Multi-Layered Autonomous Navigation Framework for the Pepper Medical Assistant

Introduction to Socially Assistive Robotics in Clinical Environments

The global healthcare sector is undergoing a profound transformation driven by the intersecting pressures of an aging population, critical shortages in clinical staffing, and the necessity for safer, infection-aware models of patient care. To mitigate these systemic strains, medical facilities are increasingly turning to intelligent automated systems capable of augmenting human workflows. Among the most promising of these innovations is the deployment of Socially Assistive Robots (SARs). Empirical research demonstrates that the physical embodiment of a humanoid robot provides measurable psychological benefits in hospital settings, particularly for vulnerable demographics such as pediatric and geriatric patients.[1] Clinical studies, such as those conducted at the UCLA Mattel Children's Hospital, reveal that interactions with human-controlled companion robots can yield a 29% increase in positive emotions and a 33% decrease in negative emotions compared to standard tablet-based video communication.[1]

The Pepper robot, developed by SoftBank Robotics, has emerged as a premier platform for these applications due to its user-friendly, human-centered design, which facilitates natural engagement through speech, gesture, and physical presence. Historically, however, the clinical utility of platforms like Pepper was severely constrained by their reliance on rigid, pre-programmed dialogue trees and statically defined behavioral scripts. Modern architectural paradigms circumvent these limitations by offloading complex processing tasks to external computational layers. By integrating Large Language Models (LLMs) for dynamic conversational generation and deploying advanced spatial mapping algorithms for physical mobility, a socially assistive robot transitions from a mere interactive kiosk into a fully autonomous healthcare companion.[1]

The Pepper Medical Assistant system encapsulates this modern paradigm through a highly synchronized, three-tier architectural framework.[1] The system must not only converse intelligently but also physically guide patients to designated medical consultation rooms, requiring the implementation of "Full Autonomous Navigation." Achieving this functionality demands a comprehensive refactoring of the system's data management, networking, and robotic control layers. This report provides an exhaustive, expert-level analysis and implementation guide for transitioning the robot's navigation from a hardcoded spatial map to a dynamic, database-driven autonomous routing system.

Architectural Paradigm and the Interoperability Challenge

Designing a cohesive software ecosystem for the Pepper robot requires navigating a significant interoperability gap.[1] The robot's native operating environment, the NAOqi OS, relies on a legacy Linux distribution strictly bound to Python 2.7. This environment lacks the computational capacity and modern library support required for contemporary artificial intelligence frameworks, asynchronous web routing, and advanced relational database management. Conversely, modern web development and AI orchestration demand Python 3.x and modern JavaScript standards.

To resolve this friction, the Pepper Medical Assistant employs a centralized "Brain" model that cleanly separates hardware execution from computational intelligence.[1] The architecture is divided into three distinct strata:

Architectural Layer	Execution Environment	Core Technologies	Primary Responsibilities
Robot Layer	Local (Pepper Hardware)	Python 2.7, NAOqi SDK	Sensor polling, Text-to-Speech (TTS), Simultaneous Localization and Mapping (SLAM), kinematic execution.
Backend Layer	External High-Performance PC	Python 3.x, Flask, SQLAlchemy, SQLite	Database management, API endpoint hosting, WebSocket routing, AI processing (Whisper/Gemini).
User Interface Layer	Pepper Chest Tablet	HTML, CSS, ES5 JavaScript	Patient interaction, dynamic directory rendering, asynchronous data

			fetching via AJAX.

The objective of implementing autonomous patient routing necessitates synchronized updates across all three layers. The spatial coordinates for medical personnel must be migrated into the Backend Layer's relational database. The User Interface Layer must dynamically retrieve this data using legacy-compatible JavaScript. A middleware routing protocol must instantly transmit the patient's selection, and the Robot Layer must execute the physical translation through the hospital environment while managing threading constraints and dynamic obstacle avoidance.

Component 1: Backend Database and Spatial API Implementation

Relational Topology and the SQLAlchemy ORM

In early development phases, spatial coordinates representing hospital rooms are frequently hardcoded into frontend HTML or JavaScript files. While expedient for prototyping, this approach presents insurmountable maintenance challenges in a live hospital environment where medical professionals frequently rotate between consultation rooms, and departmental layouts undergo structural modifications.[1] Migrating this topological data into a centralized relational database establishes a single source of truth, ensuring that any spatial updates instantly propagate to all active robot interfaces.

The system utilizes a SQLite database (app.db) managed via the Flask-SQLAlchemy Object Relational Mapper (ORM).[1] The ORM paradigm allows developers to define the database schema utilizing Python classes, abstracting away raw SQL queries and providing built-in mechanisms for data validation and serialization. The existing hospital knowledge base already manages detailed profiles for various medical professionals across multiple departments.[1] To enable autonomous routing, the Doctor model must be extended to incorporate metrical target coordinates compatible with the robot's navigation algorithms.

Mathematical Representation of Navigational Targets

In the context of the NAOqi ALNavigation framework, a spatial destination is defined by a three-dimensional array: ``.[2]

- **X (Longitudinal Axis):** Represents the distance in meters forward (positive) or backward (negative) from the map's defined origin ``.
- **Y (Lateral Axis):** Represents the distance in meters to the left (positive) or right (negative) of the origin.
- **Theta (Orientation/Yaw):** Represents the robot's required rotational angle upon

reaching the destination, measured in radians ranging from -3.1415 to 3.1415 (-180 to 180 degrees).[4]

Python 3.x Backend Code (app.py)

The following implementation details the modifications to the Flask application. It defines the expanded Doctor model and establishes a new REST API endpoint designed to serve the topological data to the UI layer in a serialized JSON format.

Python

```python
# app.py (Backend Layer - Executed on External PC via Python 3.x)
from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
import os

# -------------------------------------------------------
# Flask Application and Database Configuration
# -------------------------------------------------------
app = Flask(__name__)

# Resolve the absolute path to the directory containing this script
basedir = os.path.abspath(os.path.dirname(__file__))

# Configure the SQLite database URI to point to app.db
app.config = 'sqlite:///' + os.path.join(basedir, 'app.db')
# Disable modification tracking to reduce memory overhead
app.config = False

# Initialize the SQLAlchemy ORM instance
db = SQLAlchemy(app)

# -------------------------------------------------------
# Relational Database Models
# -------------------------------------------------------
class Doctor(db.Model):
    """
    SQLAlchemy ORM Model representing a medical professional.
    This model has been extended from its original structure to include
    the precise metrical coordinates required for NAOqi autonomous navigation.
    """
    __tablename__ = 'doctors'
```

```
    # Primary Identification Fields
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    specialty = db.Column(db.String(100), nullable=False)

    # Autonomous Navigation Fields
    room_name = db.Column(db.String(50), nullable=False, default="General Ward")
    nav_x = db.Column(db.Float, nullable=False, default=0.0)     # X-axis distance in meters
    nav_y = db.Column(db.Float, nullable=False, default=0.0)     # Y-axis distance in meters
    nav_theta = db.Column(db.Float, nullable=False, default=0.0)  # Yaw orientation in radians

    def to_dict(self):
        """
        Serializes the model instance into a dictionary format, allowing
        the Flask framework to easily convert the data into a JSON response.
        """
        return {
            "id": self.id,
            "name": self.name,
            "specialty": self.specialty,
            "room_name": self.room_name,
            "coordinates": [self.nav_x, self.nav_y, self.nav_theta]
        }

# ---------------------------------------------------------
# REST API Endpoints
# ---------------------------------------------------------
@app.route('/api/navigation_targets', methods=)
def get_navigation_targets():
    """
    HTTP GET endpoint that retrieves all doctor records from the database
    and returns their human-readable identifiers along with their
    corresponding spatial target arrays.
    """
    try:
        # Execute a SELECT * query against the doctors table
        doctors = Doctor.query.all()

        # Utilize list comprehension to serialize the ORM objects
        results = [doc.to_dict() for doc in doctors]

        # Return the payload with standard HTTP 200 OK status
        return jsonify(results), 200
```

```
    except Exception as e:
        # Implement broad exception handling to prevent server crashes
        # during database lockouts or connection failures
        return jsonify({"error": "Internal Database Error", "details": str(e)}), 500

# -------------------------------------------------------
# Database Initialization Utility
# -------------------------------------------------------
@app.cli.command("initdb")
def init_db():
    """
    Command Line Interface (CLI) utility to instantiate the database schema
    and populate it with baseline metrical data for graduation defense testing.
    Execution: flask initdb
    """
    db.create_all()

    # Seed Data: Target coordinates must correlate with points inside the mapped.explo file
    # Theta 1.57 radians is approximately a 90-degree turn to face the room door
    doc1 = Doctor(name="Dr. Ali", specialty="Cardiology", room_name="Room 101", nav_x=2.5,
nav_y=1.0, nav_theta=1.57)
    doc2 = Doctor(name="Dr. Sara", specialty="Neurology", room_name="Room 102", nav_x=4.0,
nav_y=-1.5, nav_theta=0.0)

    db.session.add(doc1)
    db.session.add(doc2)
    db.session.commit()
    print("Database schema generated and seeded with navigational targets.")

if __name__ == '__main__':
    # Bind to 0.0.0.0 to expose the server to the local area network
    app.run(host='0.0.0.0', port=5000)
```

By decoupling the topological data from the UI and structuring it within a relational schema, the system achieves compliance with modern API-first architectural standards.[1] The /api/navigation_targets endpoint ensures that whenever the robot boots, it requests the most current facility layout, entirely eliminating the risk of navigating to obsolete waypoints.

Component 2: Frontend UI Development under Legacy Constraints

The Embedded WebKit Environment

The User Interface Layer is rendered on the tablet securely mounted to Pepper's chest. This tablet runs a proprietary, specialized Android-based operating system utilizing an older build of the WebKit browser engine.[1] A critical engineering constraint of this environment is its lack of compliance with modern ECMAScript 6 (ES6) specifications.

In contemporary web development, data fetching and asynchronous operations are typically handled using the fetch() API and Promises, alongside arrow functions (=>), block-scoped variable declarations (let and const), and template literals. Executing such syntax on the Pepper tablet will result in immediate parsing failures, rendering the application inoperable. Consequently, all JavaScript logic must be rigorously downgraded to strict ECMAScript 5 (ES5) standards.

Data retrieval must be facilitated through the legacy XMLHttpRequest (AJAX) object. Furthermore, generating DOM elements dynamically within a standard for loop presents a scoping dilemma in ES5. Because the var keyword lacks block scope, event listeners attached to dynamically created buttons will inadvertently bind to the final element of the array due to variable hoisting. This issue is resolved through the architectural pattern of Closures, specifically utilizing an Immediately Invoked Function Expression (IIFE) to capture the correct data scope for each individual iteration.

JavaScript Implementation (guide.html)

The following HTML and JavaScript implementation fulfills the requirement of replacing the hardcoded roomMap with a dynamic API call. Upon successful retrieval of the data payload, the script constructs an interactive directory. When a user taps a destination button, the UI validates the active WebSocket connection and transmits the JSON routing command directly to the middleware bridge.

HTML

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Pepper Medical Assistant - Navigation Directory</title>
  <style>
    /*
      Elder-friendly accessibility design:
      High-contrast colors, massive tap targets, and distinct active states
    */
```

```
    body {
        font-family: Arial, Helvetica, sans-serif;
        background-color: #f4f4f9;
        padding: 20px;
        text-align: center;
    }
    h1 { color: #2c3e50; font-size: 36px; }

    #status-message {
        font-size: 24px;
        color: #7f8c8d;
        margin: 20px 0;
        min-height: 30px;
    }

    #doctor-list {
        display: flex;
        flex-direction: column;
        align-items: center;
        margin-top: 30px;
    }

    .nav-button {
        background-color: #2980b9;
        color: #ffffff;
        border: 2px solid #1c5980;
        padding: 25px 40px;
        margin: 15px;
        font-size: 28px;
        border-radius: 12px;
        cursor: pointer;
        width: 85%;
        max-width: 700px;
        box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    }

    .nav-button:active {
        background-color: #1c5980;
        transform: translateY(2px);
        box-shadow: 0 1px 3px rgba(0,0,0,0.1);
    }
  </style>
</head>
<body>

  <h1>Select a Doctor to Find Their Room</h1>
  <div id="status-message">Initializing system protocols...</div>
```

```
<div id="doctor-list"></div>

<script>
  // STRICT ES5 COMPLIANCE: No let, const, arrow functions, or fetch API.

  // 1. Establish the WebSocket Connection
  // NOTE: The IP address must map to the Backend Server PC hosting ws_bridge.py
  var backendIp = "192.168.1.100";
  var wsUrl = "ws://" + backendIp + ":8765";
  var ws = new WebSocket(wsUrl);
  var statusDiv = document.getElementById("status-message");

  // WebSocket Event Handlers
  ws.onopen = function() {
    statusDiv.innerHTML = "Network link established. Downloading directory...";
    // Once the real-time link is verified, request the spatial database
    fetchNavigationTargets();
  };

  ws.onerror = function(error) {
    statusDiv.innerHTML = "Critical Error: Cannot connect to the routing bridge.";
  };

  ws.onclose = function() {
    statusDiv.innerHTML = "Network connection lost. Please contact administration.";
  };

  // 2. Fetch Topological Data via legacy AJAX (XMLHttpRequest)
  function fetchNavigationTargets() {
    var xhr = new XMLHttpRequest();
    var apiUrl = "http://" + backendIp + ":5000/api/navigation_targets";

    // Initialize asynchronous GET request
    xhr.open("GET", apiUrl, true);

    xhr.onreadystatechange = function() {
      // readyState 4 indicates the operation is complete
      if (xhr.readyState === 4) {
        if (xhr.status === 200) {
          try {
            // Parse the incoming JSON string into an array of JS objects
            var responseData = JSON.parse(xhr.responseText);
            statusDiv.innerHTML = "Please select your destination below.";
            renderDoctorButtons(responseData);
          } catch (e) {
            statusDiv.innerHTML = "Error parsing directory data.";
          }
```

```
            } else {
                statusDiv.innerHTML = "Server returned error code: " + xhr.status;
            }
        }
    };

    // Dispatch the HTTP request
    xhr.send();
}

// 3. Dynamic DOM Generation with ES5 Scope Preservation
function renderDoctorButtons(doctors) {
    var listContainer = document.getElementById("doctor-list");
    listContainer.innerHTML = ""; // Purge any placeholder content

    for (var i = 0; i < doctors.length; i++) {
        var doc = doctors[i];

        // Construct the button element
        var btn = document.createElement("button");
        btn.className = "nav-button";
        // String concatenation required (no ES6 template literals)
        btn.innerHTML = doc.name + " (" + doc.specialty + ")<br><small>" + doc.room_name + "</small>";

        // CORE ES5 PATTERN: The Immediately Invoked Function Expression (IIFE)
        // This closure traps the current iteration's 'doc' variable, preventing the
        // button click handlers from overwriting each other due to 'var' hoisting.
        (function(doctorData) {
            btn.onclick = function() {
                triggerNavigation(doctorData);
            };
        })(doc);

        listContainer.appendChild(btn);
    }
}

// 4. WebSocket Payload Transmission
function triggerNavigation(doctorData) {
    // Validate the integrity of the network socket before attempting transmission
    if (ws.readyState!== WebSocket.OPEN) {
        alert("The robot is currently offline. Please wait for reconnection.");
        return;
    }

    // Construct the standardized routing payload
    var payload = {
        type: "navigate",
```

```
            target: doctorData.coordinates,
            doctor_name: doctorData.name
        };

        // Serialize the object to a string and transmit across the socket
        ws.send(JSON.stringify(payload));

        // Provide immediate visual confirmation to the patient to prevent duplicate taps
        statusDiv.innerHTML = "Engaging motors. Pepper is computing the route to " +
doctorData.room_name + ".";
    }
  </script>
</body>
</html>
```

The meticulous adherence to ES5 paradigms ensures total stability when the UI is rendered through the ALTabletService proxy.[1] By offloading the data logic to the backend, the UI script remains lightweight, serving solely as a dynamic presentation layer and a WebSocket transmitter.

Component 3: Asynchronous Message Routing via WebSocket Bridge

The Necessity of Middleware in Robotic IPC

To facilitate instant, low-latency communication between the UI tablet and the robot's internal hardware modules, the system relies on a WebSocket bridge rather than traditional REST polling.[1] Traditional HTTP architectures require the client to continuously request updates from the server, generating unacceptable overhead and latency in robotic human-interaction scenarios. WebSockets, conversely, upgrade the standard HTTP handshake into a persistent, full-duplex TCP connection, allowing data payloads to be pushed instantaneously in either direction.[5]

However, the legacy Python 2.7 environment native to the Pepper robot cannot efficiently host a modern, high-concurrency WebSocket server.[1] To bypass this limitation, the system architecture dictates the use of an external Python 3.x router (ws_bridge.py). This middleware acts as a central message broker implementing a Publish-Subscribe (Pub/Sub) pattern. Both the UI tablet and the robot's local execution script connect to this bridge as clients. When the bridge receives a navigation payload from the tablet, it instantly broadcasts that exact payload to the robot listener.[1]

Implementation of the Asynchronous Router (ws_bridge.py)

The router leverages Python 3's asyncio framework and the third-party websockets library to manage highly concurrent IO-bound network connections without blocking threads.

Python

```python
# ws_bridge.py (Backend Layer - Executed on External PC via Python 3.x)
import asyncio
import websockets
import json
import os

# Thread-safe (in the context of the asyncio event loop) registry of all active socket connections
connected_clients = set()

async def ws_handler(websocket, path):
    """
    Primary asynchronous coroutine that handles the lifecycle of an individual WebSocket client.
    It manages registration, inbound message parsing, outbound broadcasting, and cleanup.
    """
    # 1. Connection Registration
    connected_clients.add(websocket)
    print(f" Connection established. Total active nodes: {len(connected_clients)}")

    try:
        # 2. Continuous Asynchronous Message Loop
        # This loop yields control to the event loop while waiting for network I/O
        async for message in websocket:
            try:
                # Parse the incoming string into a JSON dictionary
                data = json.loads(message)
                msg_type = data.get("type", "unknown")

                # Filter out high-frequency diagnostic/heartbeat messages to prevent log pollution
                if msg_type not in ["ping", "pong", "hello", "heartbeat"]:
                    print(f" Intercepted payload of type '{msg_type}'. Initiating broadcast...")

                # 3. Selective Broadcasting Algorithm
                # Iterate through the registry and forward the payload to all nodes
                # EXCEPT the original sender to prevent infinite echo loops.
                for client in connected_clients:
```

```
                if client!= websocket and client.open:
                    await client.send(message)

            except json.JSONDecodeError:
                # Catch malformed data without crashing the client handler
                print(f" Warning: Rejected malformed JSON payload -> {message}")

    except websockets.exceptions.ConnectionClosed as e:
        # Gracefully catch standard socket termination (e.g., client refresh or power loss)
        print(f" Connection terminated by remote host. Code: {e.code}")
    except Exception as e:
        # Catch-all for unexpected I/O errors
        print(f" Unhandled exception during active routing: {e}")
    finally:
        # 4. Strict Cleanup Protocol
        # Guaranteed to execute, removing dead sockets from the registry to prevent memory leaks
        connected_clients.remove(websocket)
        print(f" Node deregistered. Remaining active nodes: {len(connected_clients)}")

async def main():
    """
    Bootstraps the WebSocket server onto the network interface and blocks indefinitely.
    """
    # Extract the target port from environment variables, defaulting to standard 8765
    port = int(os.environ.get("WS_PORT", 8765))

    print(f" Initializing Asynchronous Router on ws://0.0.0.0:{port}")

    # Instantiate the server context manager
    async with websockets.serve(ws_handler, "0.0.0.0", port):
        # Await an unresolvable future to keep the asyncio event loop spinning
        await asyncio.Future()

if __name__ == "__main__":
    try:
        # Execute the main coroutine within the event loop
        asyncio.run(main())
    except KeyboardInterrupt:
        # Provide a clean console exit upon CTRL+C administrator intervention
        print("\n Manual shutdown sequence initiated by administrator. Halting bridge.")
```

The elegant design of this bridge lies in its simplicity; it is completely agnostic to the contents of the payload. It merely validates the JSON structure and ensures the `` array is propagated

instantly from the UI layer to the hardware layer.[1]

Component 4: Autonomous Navigation and SLAM Execution

Fundamentals of NAOqi Localization and Metrical Mapping

The core of the autonomous routing requirement is executed locally on the robot. Translating a mathematical target `` into physical wheel actuation requires the application of Simultaneous Localization and Mapping (SLAM). In the NAOqi operating system, this is managed by the ALNavigation proxy.[6]

Before the Pepper robot can be dispatched to a destination, it must be localized within a pre-recorded metrical map. This map is generated through prior exploration phases where the robot utilizes its odometry, infrared sonars, and laser sensors to construct a two-dimensional probability grid—saved as an .explo file—where pixels represent either free space, occupied space, or unknown territory.[3]

To enable autonomous navigation upon system boot, the Python 2.7 script must strictly adhere to the following ALNavigation API sequence [2]:

1. **loadExploration(path):** Loads the .explo binary data into active memory.
2. **relocalizeInMap(guess):** Injects a spatial coordinate assumption (e.g., [0.0, 0.0, 0.0]) into the SLAM engine, instructing the robot where it currently resides relative to the map's origin.
3. **startLocalization():** Engages the continuous background loop that actively compares incoming LIDAR/sonar data against the loaded map to maintain precise positional awareness.

Overcoming the Threading and Blocking Constraint

A critical architectural consideration involves the function ALNavigationProxy::navigateToInMap(target). This function is strictly **blocking**.[3] When invoked, the execution of the Python script halts entirely while the SLAM engine computes the trajectory, rotates the robot, and drives the wheels until the target is reached or an insurmountable obstacle is encountered.

If this blocking function is executed on the primary thread responsible for maintaining the WebSocket connection, the network client will be paralyzed. Modern WebSocket protocols demand the continuous processing of PING and PONG control frames to verify connection integrity (keep-alive heartbeats).[9] If the robot is driving for 30 seconds, the main thread cannot send a PONG, causing the external router to assume the robot has crashed and forcibly severe

the TCP connection.

To bypass this critical failure point, the physical movement logic must be isolated within a daemonized threading.Thread. This allows the websocket.run_forever() event loop to continuously process heartbeats on the main thread while the secondary thread manages the prolonged kinematic execution.

Kinematic Safety, Obstacles, and Error Code Evaluation

In a highly dynamic clinical environment, halls are frequently obstructed by mobile medical carts, gurneys, or personnel. The NAOqi engine constantly updates its "Safety Map" to prevent physical collisions.[6] If the robot's calculated trajectory is entirely obstructed, navigateToInMap() will abort and return an integer error code [2]:

- **Code 0 (OK):** The robot reached the exact target coordinates and respected all constraints.
- **Code 1 (KO):** Absolute failure. The path is definitively blocked by an obstacle or the target exists outside the boundaries of the mapped environment.
- **Code 2 (Constraint KO):** The destination was technically reached, but precise spatial constraints could not be fulfilled (e.g., stopping 0.5 meters short due to local clutter).

The execution script must interrogate these return codes. If the code is not 0, the robot must cease all forward momentum, verbally apologize to the patient using ALTextToSpeech, and explicitly cancel the navigation operation to prevent erratic pathfinding loops.[12]

Python 2.7 Robotic Executor Implementation (bridge.py)

The script below integrates the websocket-client library, the NAOqi proxy initialization, the SLAM localization sequence, and the threaded execution model.

Note: As per the constraints, extensive comments are included specifically to provide academic context for the student's graduation defense.

Python

```python
# bridge.py (Robot Layer - Executed locally on Pepper via Python 2.7)
# -*- encoding: UTF-8 -*-

import websocket
```

```python
import json
import threading
import time
import sys
from naoqi import ALProxy

# ====================================================================
# Global NAOqi Proxies
# These objects serve as remote procedure call (RPC) interfaces to
# the underlying C++ modules of the NAOqi operating system.
# ====================================================================
navigation_proxy = None
tts_proxy = None
motion_proxy = None
awareness_proxy = None

# ====================================================================
# Boot Sequence and SLAM Initialization
# ====================================================================
def init_robot():
    """
    GRADUATION DEFENSE NOTE:
    This function handles the critical boot sequence for autonomous routing.
    It links the physical hardware proxies and initializes the SLAM
    (Simultaneous Localization and Mapping) engine via ALNavigation.
    """
    global navigation_proxy, tts_proxy, motion_proxy, awareness_proxy

    # Internal loopback address to connect to the robot's own OS
    ROBOT_IP = "127.0.0.1"
    PORT = 9559

    try:
        print(" Initializing NAOqi hardware proxies...")
        # ALNavigation handles pathfinding and SLAM
        navigation_proxy = ALProxy("ALNavigation", ROBOT_IP, PORT)
        # ALTextToSpeech handles speech synthesis
        tts_proxy = ALProxy("ALTextToSpeech", ROBOT_IP, PORT)
        # ALMotion manages the low-level motor kinematics and stiffness
        motion_proxy = ALProxy("ALMotion", ROBOT_IP, PORT)
        # ALBasicAwareness handles automated human tracking and head movement
        awareness_proxy = ALProxy("ALBasicAwareness", ROBOT_IP, PORT)

        # Wakes the robot, turning on motor stiffness
```

```
        motion_proxy.wakeUp()

        # DEFENSE NOTE: We disable basic awareness during navigation. If the robot
        # constantly moves its head to look at people while driving, it blinds the
        # 3D depth cameras to the floor, significantly increasing collision risk.
        awareness_proxy.stopAwareness()

        # Load the pre-recorded metrical environment map
        map_path = "/home/nao/hospital_map.explo"
        print(" Loading metrical.explo map from: " + map_path)
        navigation_proxy.loadExploration(map_path)

        # DEFENSE NOTE: relocalizeInMap injects our spatial assumption into the
        # SLAM engine. We assert the robot boots precisely at the map's origin.
        print(" Assumed physical location: . Relocalizing...")
        navigation_proxy.relocalizeInMap([0.0, 0.0, 0.0])

        # DEFENSE NOTE: startLocalization engages the background thread that
        # continuously aligns LIDAR/Sonar sensor data with the loaded map.
        navigation_proxy.startLocalization()
        print(" SLAM Localization loop engaged successfully.")

    except Exception as e:
        print(" Critical failure during hardware boot: " + str(e))
        sys.exit(1)

# ======================================================================
# Threaded Execution Logic for Kinematics
# ======================================================================
def execute_navigation(target_coords, doctor_name):
    """
    GRADUATION DEFENSE NOTE:
    This function is executed inside a daemon thread. navigateToInMap is a
    blocking API call. If we ran it on the main thread, it would halt the
    WebSocket run_forever() loop. The WebSocket server would not receive
    heartbeat PONGs and would forcefully drop our connection.
    """
    try:
        # String encoding is required in Python 2.7 to prevent NAOqi C++ crashes
        announcement = "Please follow me to " + str(doctor_name) + "'s room."
        tts_proxy.say(announcement.encode('utf-8'))

        print(" Initiating path calculation to: " + str(target_coords))
```

```
        # DEFENSE NOTE: navigateToInMap actively uses the Safety Map to route
        # around dynamic obstacles. It halts the thread until complete and
        # returns an integer error code.
        result_code = navigation_proxy.navigateToInMap(target_coords)

        # Evaluate NAOqi return codes
        if result_code == 0:
            print(" Destination successfully reached.")
            tts_proxy.say("We have arrived. The doctor will see you shortly.")
        else:
            # Code 1 (KO) or Code 2 (Constraint KO) indicates path failure
            raise RuntimeError("Navigation aborted by SLAM engine. Code: " + str(result_code))

    except Exception as e:
        print(" Obstacle detected or path failed: " + str(e))

        # SAFETY MITIGATION: Instantly override all motor commands to prevent collision
        try:
            navigation_proxy.stopNavigation()
            motion_proxy.stopMove()
        except:
            pass

        # Execute the required failure-state apology
        apology = "I apologize, but my path is blocked by a hospital cart or obstacle. Navigation cancelled."
        tts_proxy.say(apology.encode('utf-8'))

# ====================================================================
# WebSocket Callbacks and Event Management
# ====================================================================
def on_message(ws, message):
    """
    Triggered by the ws_bridge router. Parses JSON and dispatches threads.
    """
    try:
        # Parse the incoming string using Python 2.7 json module
        data = json.loads(message)
        msg_type = data.get("type", "")

        if msg_type == "navigate":
            target = data.get("target")
            doc_name = data.get("doctor_name", "the doctor")
```

```
        # Validate spatial array integrity before passing to API
        if target and isinstance(target, list) and len(target) == 3:
            # Dispatch the physical execution to an isolated thread
            nav_thread = threading.Thread(target=execute_navigation, args=(target, doc_name))
            nav_thread.daemon = True
            nav_thread.start()
        else:
            print(" Payload missing valid array.")

    except ValueError:
        print(" Payload rejected. Invalid JSON formatting.")
    except Exception as e:
        print(" Unknown error in message handler: " + str(e))

def on_error(ws, error):
    print(" Network socket exception: " + str(error))

def on_close(ws):
    """
    Triggered when the network disconnects. Executes critical resource cleanup.
    """
    print(" Server connection severed. Engaging cleanup protocol...")
    try:
        # DEFENSE NOTE: stopLocalization must be called on exit to free up
        # extreme CPU overhead consumed by the SLAM engine.
        if navigation_proxy:
            navigation_proxy.stopLocalization()
            print(" CPU overhead reduced. SLAM halted.")
    except Exception as e:
        print(" Cleanup protocol failed: " + str(e))

def on_open(ws):
    """
    Triggered upon successful TCP handshake with the Python 3.x backend.
    """
    print(" Uplink to medical backend secured.")
    # Initialize hardware proxies only after network stability is verified
    init_robot()

# ====================================================================
# Process Entry Point
# ==========================================
if __name__ == "__main__":
    # Define connection parameters to the external Python 3.x PC
```

```
SERVER_IP = "192.168.1.100"
WS_PORT = "8765"
WS_URL = "ws://" + SERVER_IP + ":" + WS_PORT

print(" Attempting network handshake with " + WS_URL)

# Instantiate the WebSocket application client
ws_client = websocket.WebSocketApp(
    WS_URL,
    on_open=on_open,
    on_message=on_message,
    on_error=on_error,
    on_close=on_close
)

# DEFENSE NOTE: run_forever encapsulates an infinite while loop.
# The ping_interval argument automatically sends a lightweight TCP packet
# to the router every 30 seconds to prove the robot is still online.
ws_client.run_forever(ping_interval=30, ping_timeout=10)
```

By decoupling the kinematics from the network layer, parsing the complex navigational return parameters, and building in explicit kinematic overrides (stopNavigation()), this code guarantees that the Pepper robot operates safely within an inherently unpredictable clinical environment.[12]

Conclusions and Clinical Deployment Implications

The systematic refactoring of the Pepper Medical Assistant from a static, hardcoded configuration into an automated, database-driven spatial routing framework rectifies profound operational limitations. Hardcoded navigation poses unacceptable scalability risks in dynamic clinical environments; transitioning to the SQLAlchemy-managed app.db topology ensures instantaneous adaptability and provides a robust foundation for future API expansion.[1]

Furthermore, bridging the historical technological divide between the Python 2.7 NAOqi ecosystem and modern network infrastructure validates the three-layer architectural model.[1] The development of the asynchronous ws_bridge.py middleware elegantly overcomes the concurrency restrictions of the onboard robot hardware, while the strict adherence to ES5 XMLHttpRequest and IIFE closure patterns within guide.html ensures uncompromising stability on legacy embedded systems.[1]

Crucially, the engineering choices implemented within the robot execution script—specifically, the offloading of the blocking navigateToInMap SLAM computation to a daemonized thread—ensure that network integrity and heartbeat protocols are maintained during prolonged kinematic sequences.[9] Paired with an exhaustive evaluation of NAOqi's error

matrices to trigger immediate hardware halts upon obstacle detection, the system guarantees a fail-safe mode of operation.[3] Ultimately, this framework elevates the Pepper robot from a stationary interactive terminal into an intelligent, dynamic, and autonomous agent capable of safely navigating the complexities of modern healthcare delivery.

Works cited

1. Research Paper.pdf
2. Exploration API — Aldebaran 2.5.11.14a documentation, accessed March 29, 2026, http://doc.aldebaran.com/2-5/naoqi/motion/exploration-api.html
3. Exploration API — Aldebaran 2.5.11.14a documentation, accessed March 29, 2026, http://doc.aldebaran.com/2-5/naoqi/motion/exploration-api.html?highlight=exploration
4. ALNavigation API — Aldebaran 2.1.1.16 documentation, accessed March 29, 2026, https://www.bx.psu.edu/~thanh/naoqi/naoqi/motion/alnavigation-api.html
5. How to Build WebSocket Clients in Python - OneUptime, accessed March 29, 2026, https://oneuptime.com/blog/post/2026-02-03-python-websocket-clients/view
6. ALNavigation — Aldebaran 2.5.11.14a documentation, accessed March 29, 2026, http://doc.aldebaran.com/2-5/naoqi/motion/alnavigation.html
7. ALNavigation — Aldebaran 2.8.7.4 documentation, accessed March 29, 2026, http://doc.aldebaran.com/2-8/naoqi/motion/alnavigation.html
8. Exploration API — Aldebaran 2.8.7.4 documentation, accessed March 29, 2026, http://doc.aldebaran.com/2-8/naoqi/motion/exploration-api.html
9. How to keep connection alive with async websockets client? - Stack Overflow, accessed March 29, 2026, https://stackoverflow.com/questions/53188951/how-to-keep-connection-alive-with-async-websockets-client
10. How to run forever async websocket client? - python - Stack Overflow, accessed March 29, 2026, https://stackoverflow.com/questions/55755401/how-to-run-forever-async-websocket-client
11. ALNavigation API — Aldebaran 2.4.3.28-r2 documentation, accessed March 29, 2026, http://doc.aldebaran.com/2-4/naoqi/motion/alnavigation-api.html
12. Pepper robot behavior stops while moving - Stack Overflow, accessed March 29, 2026, https://stackoverflow.com/questions/49693847/pepper-robot-behavior-stops-while-moving
13. ALNavigation API — Aldebaran Software 2.1.0.18 documentation, accessed March 29, 2026, https://fileadmin.cs.lth.se/robot/nao/doc/naoqi/motion/alnavigation-api.html